Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —

Agenda da Formação

01. Spring Boot & Rest Básico

02. API Design, DTOs & Validação

03. Persistência com JPA & Hibernate

04. Integração Externa (TMDB)

05. Qualidade, Erros & Testes

06. Spring Security, JWT & OAuth2

07. Empacotamento & Container

08. Kubernetes Local (Minikube)

Rumos

Rumos

ACADEMIA JAVA

MyMovies: de aplicação Java a backend HTTP

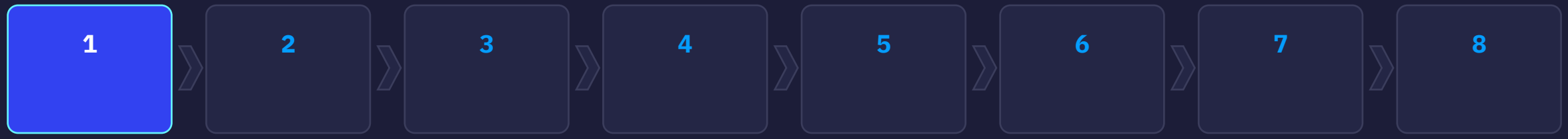
Hoje colocamos Java atrás de uma interface HTTP e devolvemos JSON.

Aula 1 • 3h30 • IntelliJ IDEA • Java 21



ROADMAP

Da primeira requisição ao serviço deployável



Rumos

C2 General

A ferramenta entra quando aparece um problema concreto

ESSÊNCIA • domínio + Java + HTTP + Spring

1	2	3	4	5	6	7
IntelliJ + JDK	Maven (mvnw)	Spring Boot	Bruno /browser	Git	Docker	CI/CD + Kubernetes
ESCREVER	**CONSTRUIR**	**EXECUTAR**	**CHAMAR**	**VERSIONAR**	**EMPACOTAR**	**ENTREGAR**
editar e depurar	dependências e build	subir a app local	ver request e response	histórico e equipe	mesma imagem em cada lugar	automatizar e operar

Git acompanha as mudanças desde o primeiro exercício — não só no fim.

REGRA DIDÁTICA

problema → ferramenta → resultado observável

O objetivo é desenvolver e compreender a API; as ferramentas só reduzem trabalho repetitivo e risco.

Rumos

C2 General

Não é necessário aprender todas as ferramentas de uma vez

AGORA • PRIMEIRO ENDPOINT

1. **Abrir e editar**
   JDK + IntelliJ
2. **Executar**
   `Run ou ./mvnw spring-boot:run`
3. **Chamar GET**
   browser; Bruno guiado
4. **Guardar checkpoint**
   `git status → add → commit`

QUANDO O FLUXO CRESCER

5. **Validar mais**
   Bruno: coleção + ambiente
6. **Construir com confiança**
   `./mvnw test / package`
7. **Colaborar**
   branch → push → merge

QUANDO FORMOS ENTREGAR

8. **Empacotar igual**
   Docker: imagem → container
9. **Automatizar**
   pipeline: build → test → deploy
10. **Operar**
    Minikube: configuração + logs

Mínimo suficiente hoje; profundidade quando houver necessidade.

C2 General

Rumos

PROBLEM

Como outro sistema chama o nosso Java?

Uma classe Java não é, por si só, uma interface de rede.

- Precisamos de um protocolo comum.
- Precisamos endereçar uma função/recursos.
- Precisamos trocar uma representação compreensível fora da JVM.



Rumos

C2 General

Uma chamada local vira endpoint quando cruza a rede

CHAMADA LOCAL

```
movieService.list()
```

1 O chamador já está dentro da aplicação
2 Usa referências e memória da mesma JVM
3 O método não tem endereço de rede

ENDPOINT DE REDE

```
GET  /api/movies
```

1 O cliente usa um endereço
2 Cliente e servidor concordam num protocolo
3 Dados precisam de uma representação: JSON

Endpoint = uma porta de entrada identificável para executar uma capacidade do backend.

PARE E EXPLIQUE • O que precisa existir para outro computador chamar list()?

Rumos
C2 General

CONCEPT

HTTP: request entra, response sai

REQUEST

```
GET  /api/movies
Host: localhost:8080
Accept:
application/json
```



RESPONSE

```
200 OK
Content-Type: application/json
[{ "id": 1, "title": "Interstellar"
}]
```


C2 General

Rumos

CONCEPT

Um endpoint é um contrato em quatro partes



QUANDO O REQUEST CHEGA

Spring encontra a rota → chama o método Java → serializa o retorno

TESTE MENTAL • Se método ou caminho muda, o cliente precisa conhecer o novo contrato.

Rumos

C2 General

CONCEPT

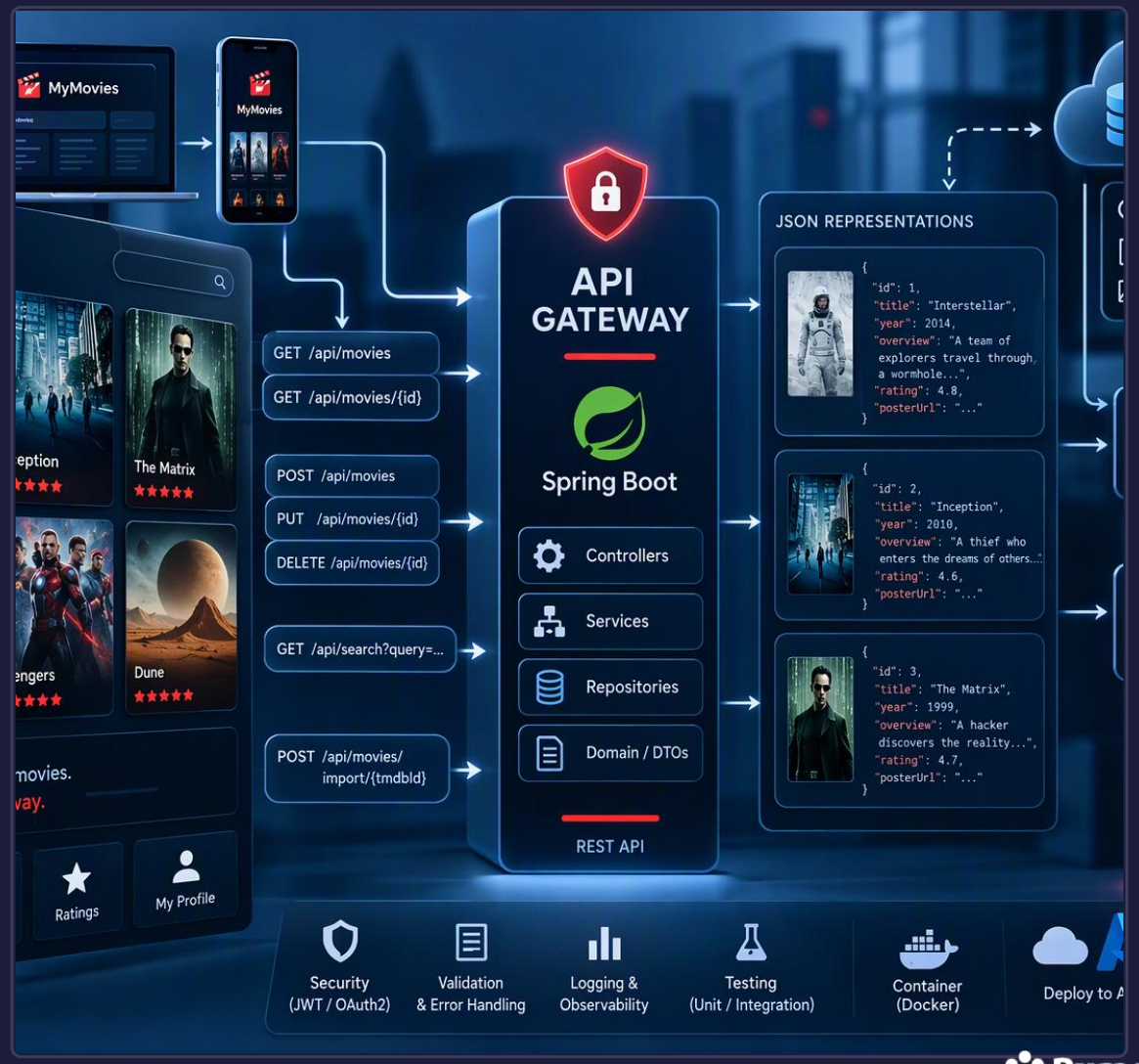
REST: pense primeiro em recursos

movie é o recurso

`/api/movies`

`/api/movies/1`

O JSON é uma representação do recurso, não o recurso em si.



C2 General

Rumos

CONCEPT

Java, Spring e Spring Boot não são a mesma coisa

JAVA

Linguagem e plataforma

Escrevemos classes e lógica.

SPRING

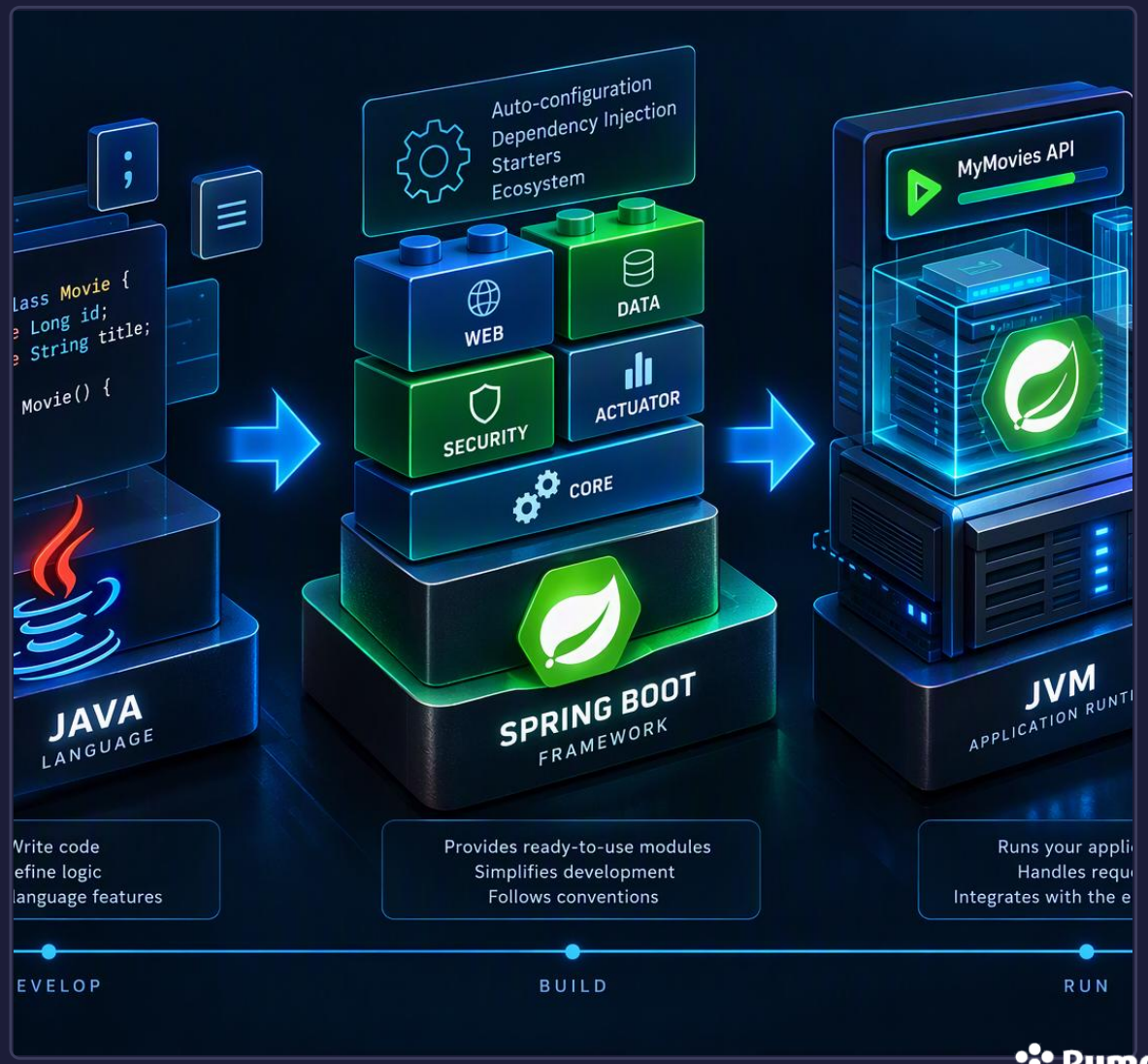
Framework/ecossistema

Organiza componentes e integrações.

SPRING BOOT

Bootstrap + convenções

Reduz configuração para executar a aplicação.



C2 General

CONCEPT

Spring cuida do encanamento para focarmos na regra

VOCÊ ESCREVE

1

Controller, serviço e regra de negócio

SPRING CONECTA

2

Cria objetos, injeta dependências e roteia

BOOT INICIA

3

Servidor embutido, configuração e operação

POR QUE USAR?

- Menos código repetitivo de infraestrutura
- Integrações consistentes: web, dados, segurança
- Componentes substituíveis e testáveis

O PREÇO DA FACILIDADE

- Convenções escondem alguns detalhes
- O ecossistema é grande
- Ainda precisamos entender HTTP e Java

C2 General

Rumos

CHOICES

Spring Boot é uma opção — não o único caminho Java

Servlet API + container	**Base web tradicional**	Mais controlo; mais configuração e infraestrutura explícita.
Jakarta REST + runtime	**Padrão Java para REST**	APIs padronizadas; implementação/runtime fornece a execução.
Spring Boot	**Convenção + grande ecossistema**	Servidor embutido, starters, auto-configuração e operação.
Quarkus • Micronaut • Helidon	**Alternativas cloud-native**	Modelos próprios para DI, configuração, startup e imagem nativa.

Escolha por **equipa, ecossistema, operação e restrições** — não por moda.

Rumos

C2 General

DIAGRAM

O caminho da primeira requisição



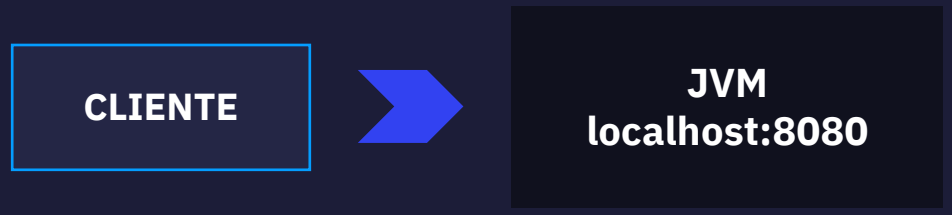
C2 General

Rumos

DIAGRAM

Da máquina local a vários servidores, a URL pode continuar igual

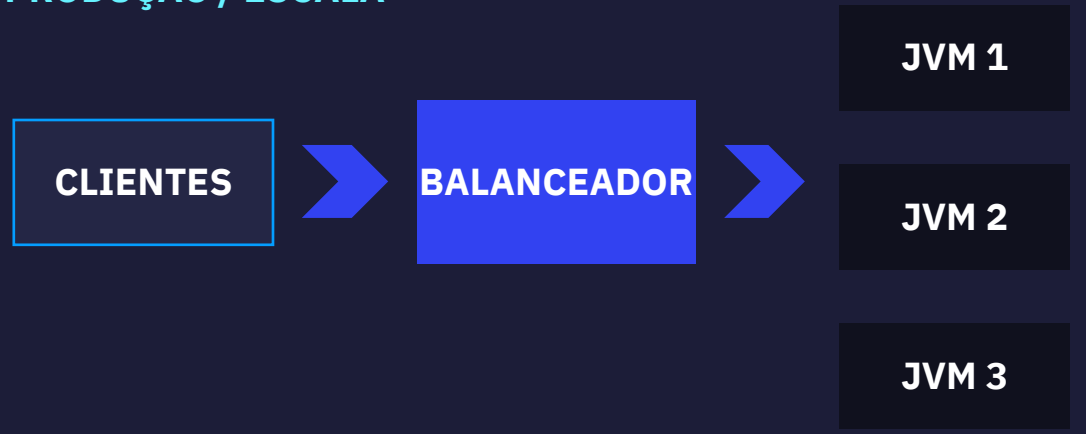
AULA / LOCAL



Uma instância • uma memória • um processo

A lista em memória funciona porque
todos os requests chegam à mesma JVM.

PRODUÇÃO / ESCALA



Cada JVM tem memória própria.
Estado compartilhado vai para DB/cache.

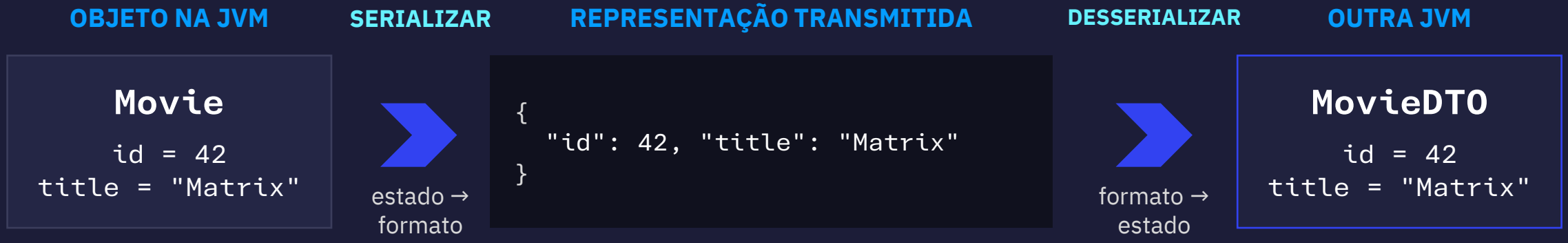
DB / CACHE

Stateless = qualquer instância pode atender o próximo request sem depender da memória da anterior.

Rumos

C2 General

A rede transporta bytes — objetos viram representação



O CONTRATO PRECISA DEFINIR

1 Campos e nomes

id, title, year...

2 Tipos e codificação

número, texto, UTF-8...

3 Versão e ausência

campo novo? null? obrigatório?

IDEIA-CHAVE • Não enviamos o objeto: enviamos uma representação que o outro lado consegue reconstruir.

Rumos

C2 General

Sockets resolveram o transporte — não o significado



O socket conecta as pontas. Cliente e servidor ainda precisam inventar e documentar o protocolo da aplicação.

TRÊS DECISÕES INEVITÁVEIS

Fronteira

Quebra de linha, tamanho ou marcador.

Representação

Texto, binário, ordem e tipos dos campos.

Falhas

Timeout, erro, repetição e versão incompatível.

TCP entrega um fluxo ordenado de bytes; não define campos nem fronteiras de mensagem.

Rumos

C2 General

RMI e CORBA fizeram a chamada remota parecer local

JAVA RMI

```
movieService.findById(42)
```

stub + serialização Java

parâmetros e retorno atravessam a rede

JVM remota

bom encaixe Java ↔ Java

CORBA

```
Movie findById(in long id);
```

IDL + ORB + marshalling

interface comum e tradução de tipos

Sistemas heterogéneos

Java ↔ C++ ↔ outros

A vantagem: menos código de rede. O risco: a sintaxe parece local, mas latência, timeout e falha continuam remotos.

PARE E EXPLIQUE • O que pode acontecer entre chamar o método e receber o retorno?

Rumos
C2 General

SOAP/WSDL formalizou o contrato — com mais verbosidade

1 WSDL DESCREVE

Serviço

`MovieService`

Operação

`GetMovie`

Tipos

`MovieRequest / Movie`

2 SOAP TRANSPORTA A MENSAGEM

```
<soap:Envelope>
  <soap:Body>
    <GetMovie>
      <id>42</id>
    </GetMovie>
  </soap:Body>
</soap:Envelope>
```

3 O CLIENTE VALIDA

GANHOS

Contrato formal
Tipos explícitos
Clientes gerados

CUSTOS

XML maior
Mais infraestrutura
Mais cerimónia

Evolução do problema: de “como enviar bytes?” para “como publicar um contrato interoperável e verificável?”.

Rumos
C2 General

PROTOCOLS

Um backend pode funcionar sem HTTP — mas não sem protocolo

HTTP E DERIVADOS	**REST sobre HTTP • gRPC usa HTTP/2 • WebSocket começa com HTTP Upgrade**	Ótimo para APIs web, streaming e integração síncrona.
TCP / RPC DIRETO	**Java RMI/JRMP • socket TCP • protocolo binário próprio**	Sem HTTP; cliente e servidor precisam conhecer o mesmo contrato.
MENSAGENS / EVENTOS	**AMQP • MQTT • protocolo Kafka • broker/filas**	O produtor publica; o consumidor processa depois — sem chamada direta.

Não confundir: **REST** é estilo arquitetural; **HTTP/AMQP/MQTT/JRMP** são protocolos; **JSON/Protobuf** são formatos.

PERGUNTA • Se o emissor não puder esperar pela resposta, que modelo faz mais sentido?

C2 General

TIMELINE

A integração mudou — hoje combinamos APIs, RPC e eventos

1990s

1

Sockets • RMI • CORBA

"objeto remoto"
e contratos binários

2000s

2

SOAP • WSDL

contrato formal;
muitas vezes sobre HTTP

2010s

3

REST • JSON

simplicidade e
compatibilidade web

HOJE

4

REST + eventos + gRPC

mistura orientada
a cada necessidade

O QUE NÃO MUDOU

Endereço • contrato • serialização • timeout • autenticação • tratamento de falhas

RMI continua no Java; os módulos CORBA saíram do JDK 11. Tecnologias antigas podem existir em legados.

C2 General

CODE

O menor controller útil

CODE

```java
@RestController
@RequestMapping("/api/movies")
class MovieController {

    @GetMapping
    List<Movie> list() {
        return movies;
    }
}
```

@RestController recebe requests HTTP

@RequestMapping prefixo do recurso

@GetMapping mapeia GET ao método

C2 General

IntelliJ: quatro pontos para não perder tempo

1

JDK 21

Project SDK

2

Maven

pom.xml importado

3

Run

MyMoviesApplication

4

Logs

porta e startup

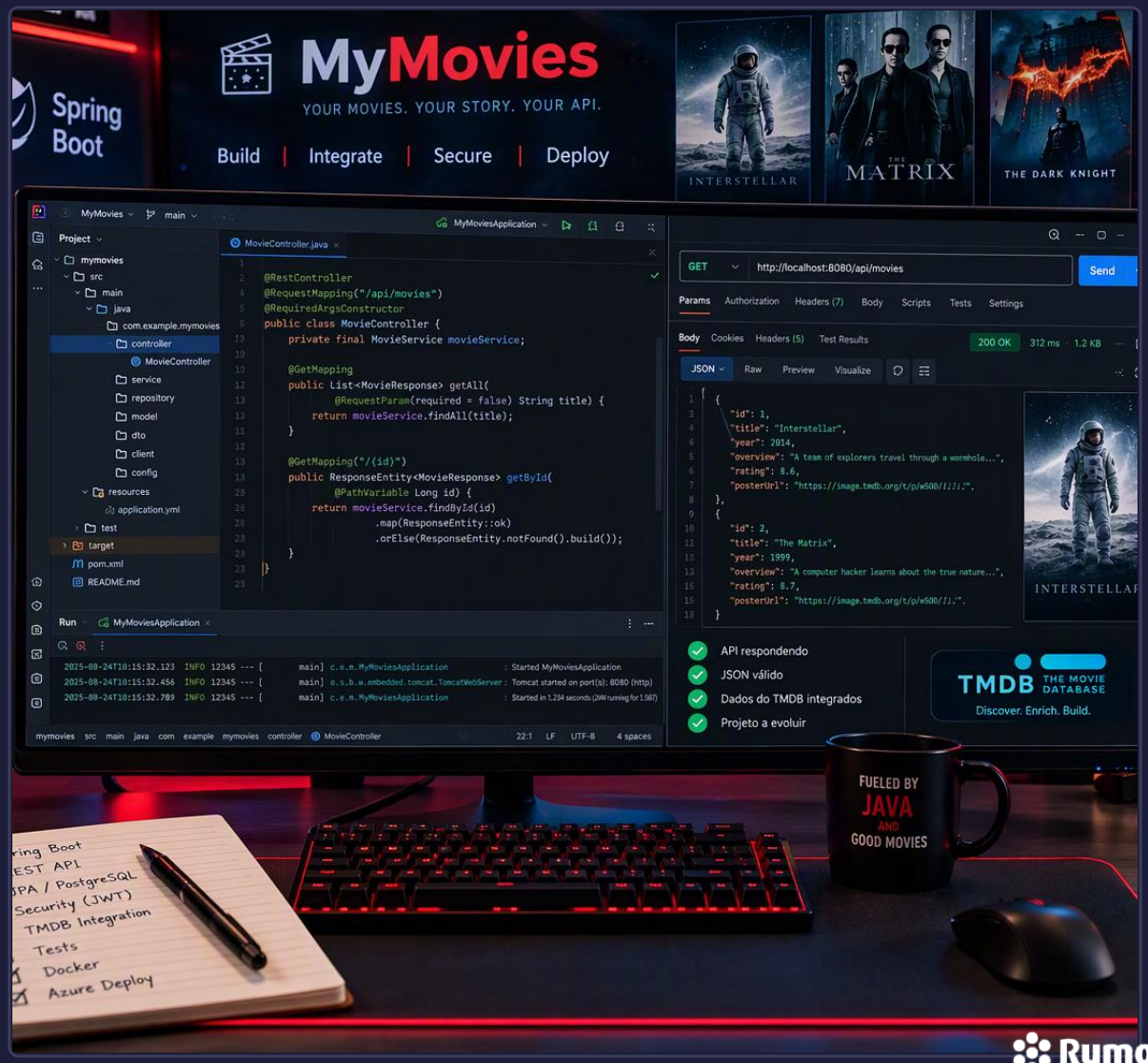
C2 General

Task: GET /api/movies

- Execute MyMoviesApplication.
- Chame GET /api/movies.
- Coloque breakpoint no método list().
- Repita o request e siga o objeto até o JSON.

Task

Critério: 200 + JSON de filmes



C2 General

CONCEPT

Path variable ou request parameter?

IDENTIFICAR UM RECURSO

`/api/movies/1`

```
@PathVariable Long id
```

FILTRAR / MODIFICAR A CONSULTA

`/api/movies?title=matrix`

```
@RequestParam String title
```

CODE

Recurso por id: só o necessário

CODE

```
@GetMapping("/{id}")
Movie byId(@PathVariable Long id) {
    return movies.stream()
        .filter(m -> m.id().equals(id))
        .findFirst()
        .orElse(null);
}
```

Hoje

- Ler id do path
- Procurar em memória
- Retornar objeto

Depois 404 e error handling serão tratados de forma adequada noutra etapa.

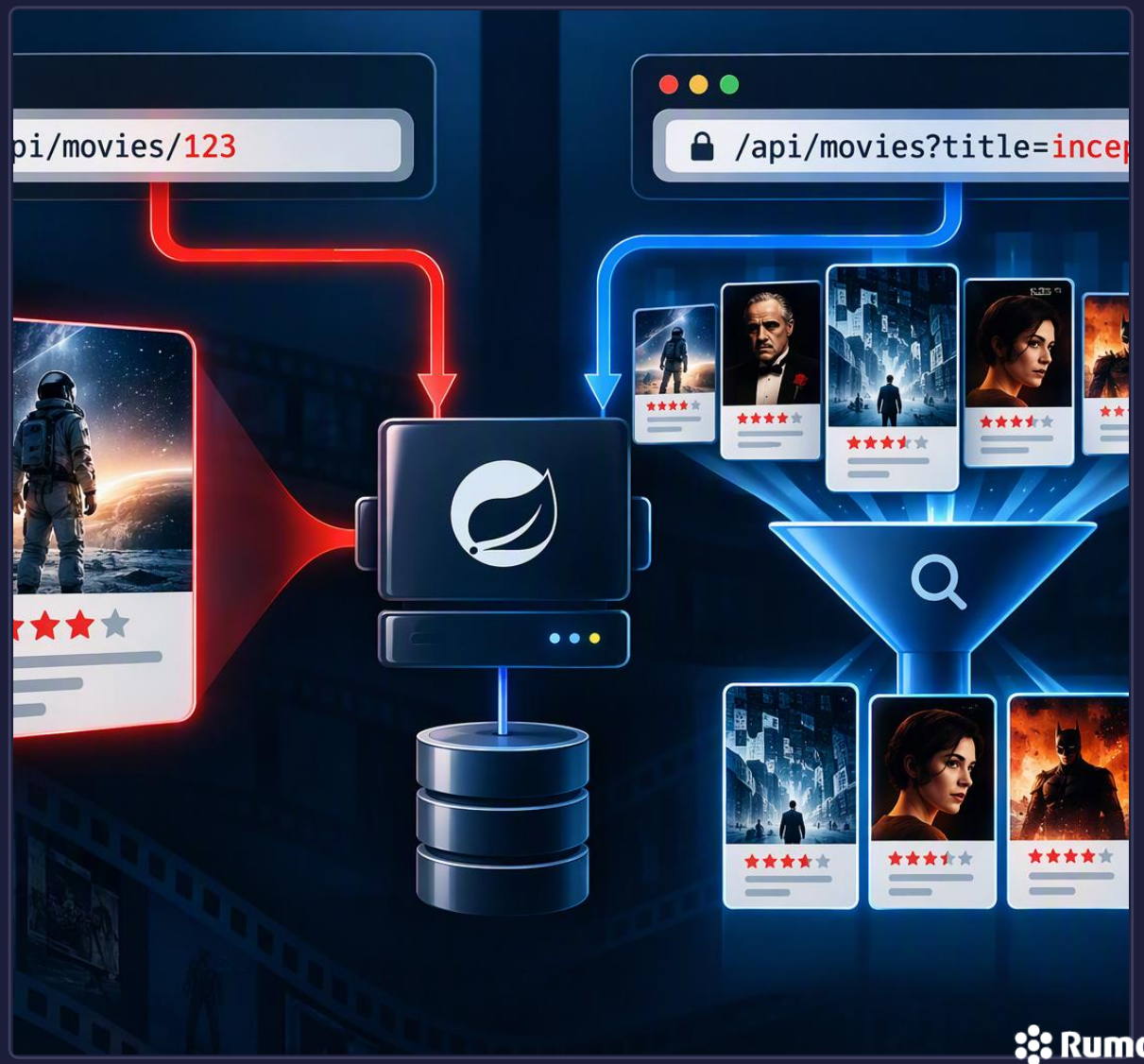
C2 General

TASK

Task: GET /api/movies/{id}

- Adicione @GetMapping("/{id}").
- Execute /api/movies/1.
- Inspecione id no debugger.
- Execute /api/movies/999 e observe a lacuna.

A lacuna é parte do roteiro: APIs previsíveis chegam na Aula 5.



C2 General

Java refresher: leia, não reaprenda

```
movies.stream()
      .filter(m ->m.id().equals(id))
      .findFirst()
      .orElse(null);
```

Stream — percorre uma coleção de forma declarativa

filter — mantém itens que satisfazem a condição

findFirst — produz Optional; aqui simplificado para a aula

Rumos
C2 General

IA como tutor: peça explicação, não a solução

> “Explique o caminho desta requisição HTTP até o JSON. Não gere código.”

- Se vier código completo, rejeite e reformule.
- Compare a resposta da IA com o debugger.
- Você deve conseguir explicar cada linha que mantém.



C2 General

Checkpoint: aula-01-rest

Antes de terminar, prove o estado do repositório.

- ✓ Aplicação inicia no IntelliJ
- ✓ GET /api/movies retorna JSON
- ✓ GET /api/movies/1 retorna um Movie
- ✓ Você explica HTTP → controller → JSON

`checkpoint/aula-01-rest`

Rumos
C2 General

Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —